PG-Finder

Backend Testing Report

Date: 22 October 2026 **Stack:** Node.js, Express, MongoDB, Socket.IO, Razorpay

1. About this report

PG-Finder is a platform where tenants search for PGs and hostels, chat with owners, book a bed and pay a token amount online. Owners manage listings and booking requests, and an admin approves listings and manages users. This report covers the main backend APIs, the ones where a bug would hurt the most: login and roles, search, listings, bookings, payments, chat and admin moderation.

Smaller routes such as wishlist, notifications, health check and the review list are simple reads and were left out to keep the report focused.

2. How the tests are done

Requests are sent through Postman against a separate test database, and chat is checked with a small socket.io-client script. Payments use Razorpay test keys and test cards. The data set has one admin, two owners, two tenants and a few PGs in different states (approved, pending, rejected), plus one room that is already full. Two owners and two tenants are needed so that checks like "can Owner A touch Owner B's PG" can actually be tried.

For each case I check the HTTP status, the JSON that comes back, and what really changed in MongoDB. The last one matters because a 200 can hide a bed count that went up twice.

3. Test cases

3.1 Authentication

ID	API	What is tested	Expected result
AUTH-01	POST /api/auth/register	Register a tenant with valid name, email, phone, password	201. User saved with a hashed password; the hash is not in the response
AUTH-02	POST /api/auth/register	Register again with the same email	Refused with a clear message, no duplicate user
AUTH-03	POST /api/auth/register	Send a bad email and a 3-character password	400 from Zod validation naming the bad fields
AUTH-04	POST /api/auth/register	Send role = admin	Refused. Admin can't be created from the public form
AUTH-05	POST /api/auth/login	Correct email and password	200 and an HTTP-only cookie
AUTH-06	POST /api/auth/login	Wrong password	401 with a generic message that doesn't say whether the email exists
AUTH-07	POST /api/auth/login	Sixth attempt within one minute	429 from the auth rate limiter
AUTH-08	GET /api/auth/me	Call with a valid cookie, then with none	Profile the first time, 401 the second
AUTH-09	POST /api/auth/logout	Log out, then call /me	Cookie cleared, /me returns 401

3.2 Role-based access

ID	API	What is tested	Expected result
ROLE-01	GET /api/dashboard/owner	Call as a tenant	403
ROLE-02	GET /api/admin/stats	Call as a tenant and as an owner	403 for both
ROLE-03	Any protected route	Call without logging in	401, never a 500

3.3 PG search

ID	API	What is tested	Expected result
SRCH-01	GET /api/pgs/search	Search with no filters	Paginated list with approved PGs only
SRCH-02	GET /api/pgs/search	Filter by city, gender and a rent range	Only matching listings come back
SRCH-03	GET /api/pgs/search	Send lat, lng and radius	Listings inside the radius, nearest first
SRCH-04	GET /api/pgs/:id	Open a pending or rejected PG as a public user	Not visible to the public
SRCH-05	GET /api/pgs/:id	Use a malformed id, then a valid id that doesn't exist	400, then 404

3.4 Owner listings and rooms

ID	API	What is tested	Expected result
OWN-01	POST /api/owner/pgs	Create a PG with valid data	Created with status pending
OWN-02	POST /api/owner/pgs	Create a PG with missing title and out-of-range coordinates	400 with field errors
OWN-03	PUT /api/owner/pgs/:id	Owner A edits a PG that belongs to Owner B	403 or 404, and the PG is unchanged
OWN-04	POST /api/owner/pgs/:id/rooms	Add a room with 0 beds or a negative rent	400
OWN-05	PUT /api/owner/pgs/:id/rooms/:roomId	Set occupiedBeds higher than totalBeds	Rejected
OWN-06	POST /api/owner/pgs/upload	Upload 11 images, then a non-image file	Both rejected. Up to 10 images are accepted

3.5 Bookings

ID	API	What is tested	Expected result
BOOK-01	POST /api/bookings	Tenant requests an available room	Booking is pending and the owner is notified
BOOK-02	POST /api/bookings	Request a room that is already full	Rejected
BOOK-03	POST /api/bookings	Same tenant requests the same room twice	Second request blocked
BOOK-04	PATCH /api/bookings/:id/approve	Owner approves a pending booking	Status approved, holdExpiresAt about 24 hours ahead, tenant notified
BOOK-05	PATCH /api/bookings/:id/approve	A different owner tries to approve it	403
BOOK-06	PATCH /api/bookings/:id/reject	Owner rejects with a reason	Status rejected, reason saved, tenant notified
BOOK-07	PATCH /api/bookings/:id/cancel	Tenant cancels their own booking, then someone else's	First works and history is updated, second is 403

3.6 Payments (Razorpay test mode)

ID	API	What is tested	Expected result
PAY-01	POST /api/payments/create-order	Create an order for an approved booking	Razorpay order created and the amount matches the token amount
PAY-02	POST /api/payments/create-order	Create an order for a pending booking	Rejected
PAY-03	POST /api/payments/verify	Send the correct signature	Payment paid, booking confirmed, occupied beds up by one
PAY-04	POST /api/payments/verify	Send a tampered signature	Rejected and the booking is untouched
PAY-05	POST /api/payments/verify	Send the same verification twice	Second call changes nothing, beds not counted twice
PAY-06	POST /api/payments/webhook	Valid signature, payment captured	Processed. Booking confirmed even if /verify was never called
PAY-07	POST /api/payments/webhook	Wrong or missing signature	Rejected
PAY-08	POST /api/payments/refund	Owner refunds a paid, cancelled booking	Refund goes through, payment marked refunded
PAY-09	POST /api/payments/refund	Refund more than the amount paid	Rejected

3.7 Chat

ID	API	What is tested	Expected result
CHAT-01	POST /api/chat/start	Start a chat for a PG, then start it again	Same conversation is returned the second time
CHAT-02	POST /api/chat/conversations/:id/messages	Send an empty message, then a normal one	Empty rejected, normal saved
CHAT-03	GET /api/chat/conversations/:id/messages	A user who isn't part of the chat asks for the history	403
CHAT-04	Socket: connect	Connect with a valid JWT, then with none	Connects, then refused
CHAT-05	Socket: send_message	Tenant sends a message while the owner is connected	Saved in the database; owner gets new_message and an updated unread count

3.8 Admin

ID	API	What is tested	Expected result
ADM-01	PATCH /api/admin/pgs/:id/status	Approve a pending PG	Becomes public and the owner is notified
ADM-02	PATCH /api/admin/pgs/:id/status	Reject a PG with a reason	Hidden from the public, reason stored
ADM-03	PATCH /api/admin/users/:id/block	Block a user who is logged in	They can't log in again and their current session stops working

4. Parts that need extra attention

Double confirmation. A booking can be confirmed by /payments/verify and by the webhook. If both arrive together the bed count could go up twice (PAY-05, PAY-06).

Public webhook. It has no login and no rate limit, so the signature check is the only protection. It also needs the raw body, so it must be mounted before the JSON parser (PAY-07).

Auth limit of 5 per minute. On a shared college or hostel network, real users could lock each other out. It also slows testing down, so wait a minute between runs.

Owner-to-owner access. A role check doesn't prove a record belongs to that owner, so OWN-03 and BOOK-05 try to cross that line on purpose.

Blocked users. A blocked user may still hold a valid token or an open socket (ADM-03).